

IST608 — Advanced Topics in Database Management (Resit 2024/2025)

Course: IST608 · **Institution context:** University of Buea-style paper

These are **model answers** for study; wording follows common database texts (file organizations, indexing, transactions, storage).

Section A — Answer all questions

Question 1 (19 marks)

(a) Unordered (heap) file

Assumptions: Records are stored in insertion order; no ordering key is maintained on disk; blocks are read/written as units.

(i) Searching for a record

- **Process:** Unless a separate access structure exists, search is **linear**. The DBMS reads blocks sequentially (or uses a known allocation map) and scans records inside each block until the target is found or the file ends.
- **Efficiency:** Average case about **half the file** must be examined for a matching record if it exists and is unique; worst case **all blocks**. Cost is **$O(b)$** block accesses in the absence of an index — poor for large files.

(ii) Inserting a new record

- **Process:** Append the record to the **last block** if space remains; otherwise allocate a **new block**, place the record there, and update file metadata (e.g., end-of-file pointer).
- **Efficiency:** Typically **1–2** block accesses (read last block, possibly write it, write new block) — **very efficient** for append-only insertion. No reordering is required.

(iii) Deletion of a record

- **Process:** **Find** the record (same as (i)), then either:
 - **Physical delete:** shift following records to close the gap (expensive in unordered files because it is still a scan + possible block compaction), or
 - **Logical delete:** mark the slot with a **deletion marker/tombstone** (as in your EMPLOYEE record layout) and optionally reclaim space later.
 - **Efficiency:** Finding the record is **$O(b)$** like search. Physical compaction can be **$O(b)$** in the worst case. Logical delete after locate is **$O(1)$** per record update but wastes space until reorganization.
-

(b) Ordered file (sequentially ordered on a key, e.g., Ssn)

(i) Searching for a record

- **Process:** Use **binary search** on blocks (locate the block whose key range contains the search key), then scan within the block (or binary search in-block if records are ordered and dense).
- **Efficiency:** About $\lceil \log_2 b \rceil$ block accesses for the binary search on b blocks, plus **1** block to confirm/read the record — **much better** than linear scan for large b .

(ii) Inserting a new record

- **Process:** **Locate** the correct position by key order (binary search), then **make room** inside the block; if the block is full, **split** or **overflow** handling is required (shift records in subsequent blocks or use overflow chains — depends on implementation).
- **Efficiency:** Locating is $O(\log b)$ block reads; maintaining physical order may require **many block reads/writes** (shifting/splitting) — **insertion is expensive** compared to heap files.

(iii) Deletion of a record

- **Process:** Find the record (binary search + local scan), remove it, then **compact** within the block or **merge** blocks to preserve order without leaving unusable gaps (or mark deleted and periodically reorganize).
- **Efficiency:** Find is $O(\log b)$; physical compaction can again touch **many blocks** — **costly**, similar in spirit to ordered insertion.

(c) EMPLOYEE file — record size, blocking factor, blocks

Given

Symbol	Meaning	Value
B	Block size	512 bytes
P	Block pointer	6 bytes
PR	Record pointer	7 bytes
r	Number of records	30,000

Organization **Unspanned** each record wholly in one block

Field sizes (bytes): Name 30, Ssn 9, Department_code 9, Address 40, Phone 10, Birth_date 8, Sex 1, Job_code 4, Salary 4, deletion marker 1.

(i) Record size R

Rule: Sum all user fields plus any record-level overhead given (here, the deletion marker).

$$[R = 30 + 9 + 9 + 40 + 10 + 8 + 1 + 4 + 4 + 1 = 116 \text{ bytes}]$$

(ii) Blocking factor bf and number of file blocks b (unspanned)

Rules (unspanned):

- A record must not span two blocks, so only **whole** records fit in a block.

- **Blocking factor:** maximum records per block

$$[bf = \left\lfloor \frac{B}{R} \right\rfloor]$$
- **Number of blocks:**

$$[b = \left\lceil \frac{r}{bf} \right\rceil]$$

Working

$$[\frac{B}{R} = \frac{512}{116} = 4.413\dots]$$

$$[bf = \left\lfloor 4.413\dots \right\rfloor = 4 \text{ records/block}]$$

$$[\frac{r}{bf} = \frac{30,000}{4} = 7,500 \text{ exactly}]$$

$$[b = \left\lceil 7,500 \right\rceil = 7,500 \text{ blocks}]$$

Assumptions stated: No block header overhead (not specified); unspanned packing uses **floor** of B/R ; file is contiguous in the sense that we only count data blocks needed for r records.

(d) Primary index on S_{sn} (file ordered by S_{sn})

Standard assumption (primary index on ordering key of an ordered file): the index is **sparse (non-dense): one index entry per data block**, storing the **first (lowest) key** in that block and a **pointer to the block** (uses P , not PR).

From part (c): $b = 7,500$ data blocks, $B = 512$, $P = 6$, key $S_{sn} = 9$ bytes.

(i) Index blocking factor bf_{ri}

Index entry size R_i (sparse primary):

$$[R_i = \text{key size} + P = 9 + 6 = 15 \text{ bytes}]$$

Rule: Same as data blocking, but with index entry length:

$$[bf_{ri} = \left\lfloor \frac{B}{R_i} \right\rfloor = \left\lfloor \frac{512}{15} \right\rfloor = \left\lfloor 34.133\dots \right\rfloor = 34 \text{ entries/index block}]$$

(ii) First-level index entries and first-level index blocks

Rule (sparse primary): number of first-level entries = number of data blocks:

$$[N_{\text{entries}} = b = 7,500]$$

Rule: index blocks = ceiling of entries over index blocking factor:

$$[b_1 = \left\lceil \frac{N_{\text{entries}}}{bf_{ri}} \right\rceil = \left\lceil \frac{7,500}{34} \right\rceil = \left\lceil 220.588\dots \right\rceil = 221 \text{ index blocks}]$$

(iii) Block accesses to retrieve a record by S_{sn} using the primary index

Process

1. **Binary search** the **ordered index file** of **b_1 blocks** to find the index entry whose key range covers the target **S_{sn}** .
2. Follow the **block pointer** to the **single data block** and read the record (possibly small in-block scan).

Formula (worst-case block reads for binary search on sorted blocks):

$$\lceil \log_2 b_1 \rceil$$

Working

$$\log_2(221) = \frac{\ln 221}{\ln 2} \approx 7.79 \rightarrow \lceil \log_2 221 \rceil = 8$$

Add **one** data block read:

$$\text{Total} \approx 8 + 1 = 9 \text{ block accesses (typical worst-case model)}$$

Note: If your lecturer counts a different convention (e.g., exact fractional log without ceiling, or counting root separately), the integer may differ by 1; the **method** (index binary search + one data block) is what earns marks.

If dense index were required (non-standard for “primary” on ordered file): entries = **r** , entry size might be **$9 + PR = 16$** , giving different **b_{fri}** and **b_1** . The exam hint and usual definition favor **sparse primary** as above.

Question 2 (17 marks)

(a) SQL — bank deposit and withdrawal with integrity

Assumptions

- **TransactionID** is system-generated (identity) or omitted if DB assigns it.
- **TransactionDate** uses **CURRENT_TIMESTAMP** / **GETDATE()** depending on DBMS.
- **Type** is exactly **'Deposit'** or **'Withdrawal'**.
- “Empty account” interpreted as **Balance must remain ≥ 0** after withdrawal (stricter: also reject zero balance withdrawal if “empty” means balance = 0).

1) Deposit \$100 into account 101

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountID = 101;

INSERT INTO Transactions (AccountID, TransactionDate, Amount, Type)
VALUES (101, CURRENT_TIMESTAMP, 100, 'Deposit');

COMMIT;
```

2) Withdraw \$50 from account 101 — cannot withdraw if balance insufficient / empty

The withdrawal must **only** happen when `Balance >= 50` **before** the update, and the **transaction log row** must be written **only** when the balance was actually reduced (otherwise you get an orphan `Withdrawal` row or a negative balance).

Option A — SQL Server: `UPDATE ... OUTPUT` (atomic, no race in one statement)

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 50
OUTPUT inserted.AccountID, CURRENT_TIMESTAMP, 50, 'Withdrawal'
INTO Transactions (AccountID, TransactionDate, Amount, Type)
WHERE AccountID = 101 AND Balance >= 50;

COMMIT;
```

If `Balance < 50`, **zero rows** are updated and **nothing** is inserted into `Transactions`.

Option B — Procedural rowcount check (SQL Server style)

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 50
WHERE AccountID = 101 AND Balance >= 50;

IF @@ROWCOUNT = 1
    INSERT INTO Transactions (AccountID, TransactionDate, Amount, Type)
    VALUES (101, CURRENT_TIMESTAMP, 50, 'Withdrawal');

COMMIT;
```

Option C — Constraint + safe update (portable idea)

Add `CHECK (Balance >= 0)` on `Accounts`. Attempting `UPDATE ... SET Balance = Balance - 50` without the `WHERE Balance >= 50` guard could **fail** the constraint; the exam expects the `WHERE Balance >= 50` guard so the update is a **no-op** when funds are insufficient (no error, no change).

(b) Concurrency control

What is concurrency control?

It is the set of **protocols and mechanisms** (schedulers, locks, timestamps, validation, etc.) that coordinate **simultaneous transactions** so that their interleaved reads/writes preserve **correctness** (e.g., **serializability** or chosen isolation level) and **consistency** of the database.

Why necessary?

Without it, interleaving causes **lost updates**, **dirty reads**, **unrepeatable reads**, and **anomalies on constraints**

(e.g., two withdrawals both seeing “enough balance”). Concurrency control **orders or restricts** conflicting operations so invariants (like non-negative balance) hold.

Types of techniques (with examples)

1. Two-Phase Locking (2PL)

Transactions acquire locks before use; **growing phase** then **shrinking phase** (no new locks after first release). Example: **T1** locks account A for write; **T2** waits — prevents lost update.

2. Timestamp ordering (TO)

Each transaction has a timestamp; reads/writes are allowed only if order matches **Thomas Write Rule** / read rules; otherwise **abort/restart**. Example: reject a write that is “too late” relative to a younger transaction’s read.

3. Optimistic / validation (multi-version or single-version)

Execute in **private workspace**, **validate** at commit that no conflicts occurred; else **rollback**. Example: low-contention workloads with rare conflicts.

4. Multi-Version Concurrency Control (MVCC)

Keep versions; readers see a **snapshot** as of start time; writers create new versions. Example: PostgreSQL-style snapshot isolation reduces read–write blocking.

5. Deadlock handling (often paired with locking)

Wait-for graph detection or **timeout**; **victim selection** and rollback.

(Listing 3–4 with crisp definitions + one sentence example each is usually sufficient in an exam.)

Section B — Answer EITHER Question 3 OR Question 4

Question 3 — Virtualization, disk/LVM, disk metrics

(a)(i) Server virtualization (hypervisor; before vs after)

- **How it is achieved:** A **hypervisor** (Type 1 “bare-metal” or Type 2 “hosted”) runs **multiple guest OS instances** on the same physical machine. The hypervisor **virtualizes CPU, memory, I/O**, and presents **virtual devices** to each guest.
- **Role of hypervisor:** Schedules VMs on real CPUs, **traps privileged instructions**, maps **guest physical memory** to **host physical memory**, and mediates device access (emulated or paravirtualized drivers, SR-IOV in some cases).
- **Before vs after:**
 - **Before:** one OS stack per server; resources often **underutilized**; scaling meant buying more hardware.
 - **After:** **higher utilization**, **isolation** between tenants, **faster provisioning** of new servers (VM templates), **live migration** for maintenance — at the cost of **overhead** (CPU/memory for virtualization) and **operational complexity**.

(a)(ii) Memory virtualization

- **Concept:** Each process/VM sees a **virtual address space**; the **MMU** (with **page tables**, often **nested page tables** for VMs) translates to **physical frames**, optionally backed by **disk (swap/page file)**.

- **Mechanisms: Demand paging, TLB, ballooning** (reclaim from guests), **memory overcommit** (careful sharing/deduplication in some systems).

(b)(i) Physical vs logical volumes

- **Physical volume (PV):** a disk or partition the LVM can use as raw capacity.
- **Logical volume (LV):** a **virtual disk** carved from a **volume group**, appearing as a block device to the OS/filesystem — **location-transparent** to upper layers.

(b)(ii) Partitioning vs concatenation

- **Partitioning:** divide one physical disk into **separate regions** (slices) treated independently.
- **Concatenation / spanning:** join **multiple disks or extents** end-to-end so they appear as **one larger linear address space** (striping is a different pattern — parallelism).

(b)(iii) Role of Logical Volume Manager (LVM)

- Pools PVs into **volume groups (VG)**; creates resizable **LVs**; supports **snapshots, migration**, and flexible allocation without repartitioning apps manually.

(c) Disk metrics

- **Seek time:** time to **move heads** to the target cylinder/track (dominated by mechanical motion on HDDs).
 - **Rotational latency:** wait for the correct sector to rotate under the head (~ **half a rotation** on average).
 - **Data transfer rate:** bytes/sec once the head is on-track and reading/writing (interface + media rate).
 - **Most critical for typical random I/O: seek time + rotational latency** usually dominate **small random reads/writes** on HDDs; **sequential throughput** is more transfer-bound.
-

Question 4 — Intelligent storage, cache, security

(a) Roles of front end, cache, back end

- **Front end:** host connectivity (**ports**), protocol handling (SCSI/FC/iSCSI/NVMe-oF), **front-end controllers** — presents **LUNs/volumes** to hosts and accepts I/O commands.
- **Cache:** fast **DRAM (and sometimes NVMe)** holding hot data/metadata; absorbs bursts, coalesces writes, enables **read hits** and **write buffering**.
- **Back end:** **back-end controllers** and **disk ports** manage **physical disks**, RAID parity/rebuild, **destaging** from cache to disk.

(b) Cache on read/write — hits, misses, prefetch, write-back vs write-through

- **Cache hit:** requested data is in cache → **low latency**, no disk read (read hit) or deferred disk write (write hit to cached block).
- **Cache miss:** data not in cache → must **stage from disk** (read miss) or allocate and later **flush** (write miss) — **higher latency**.
- **Prefetch (read-ahead):** predictively load **sequential** neighboring blocks after a miss → turns future accesses into **hits**; great for sequential scans; risk of **wasted bandwidth** for random workloads.
- **Write-through:** each write updates **cache and disk** immediately → **stronger durability** per write, **slower** writes.

- **Write-back (write-behind):** write acknowledged after **cache** update; disk update **delayed** in batches → **faster** writes; relies on **battery/UPS + destaging** for safety on power loss.

(c) Cache security measures

- **Cache mirroring:** duplicate cache contents to **paired modules/controllers** so a single cache failure does not lose in-flight dirty data; improves **availability**.
- **Cache vaulting:** on power loss, **dump dirty cache** to **non-volatile vault** (e.g., flash module with supercapacitor) so writes can be **replayed** safely — protects against **data loss** with write-back.

Section C — Answer EITHER Question 5 OR Question 6

Question 5 — Object-based / unified storage

Securing data against unauthorized access

- **Identity & access control:** users/apps authenticate; **RBAC/ABAC** policies on **buckets/containers**; **least privilege**.
- **Encryption: at rest** (AES on disks/objects) and **in transit** (TLS); optional **client-side** encryption for sensitive payloads.
- **Immutable / WORM / legal hold:** tamper-evident retention for compliance.
- **Auditing & logging:** access trails, anomaly detection.
- **Network isolation:** VPC endpoints, private links, firewall rules.

Why object storage is efficient for large data

- **Flat namespace + key addressing** scales to billions of objects without classic hierarchical bottlenecks.
- **REST/S3 APIs** suit **large blobs**; **erasure coding** and **wide striping** give durability with lower overhead than triple-mirror for huge scale.
- **Metadata** beside objects enables search/catalog patterns.

Typical data stored

- Media (video/audio), backups/archives, **data lake** files (Parquet/ORC), logs, static web assets, ML datasets, medical imaging — generally **unstructured/semi-structured** large files plus metadata.

Unified storage (brief): single platform exposing **block + file + object** interfaces with shared pools, simplifying management while mixing workload types.

Question 6 — Cloud deployment models and providers

(i) Deployment models

- **Public cloud:** shared provider infrastructure (multi-tenant), pay-as-you-go, fastest to provision; data resides at provider.
- **Private cloud:** cloud stack **dedicated** to one organization (on-prem or hosted); stronger control/compliance; higher fixed cost.
- **Hybrid cloud:** mix **private + public** with orchestration (burst capacity, DR, sensitive data on-prem).

- **Community cloud:** shared by a **community** with common concerns (e.g., universities, government agencies) — shared costs, tailored policies.
- (ii) **Three cloud providers and service types**
1. **Amazon Web Services (AWS):** IaaS (EC2, EBS), PaaS (RDS, Elastic Beanstalk), SaaS (WorkMail), serverless (Lambda).
 2. **Microsoft Azure:** IaaS (VMs), PaaS (Azure SQL, App Service), SaaS (Microsoft 365 integration), hybrid (Arc).
 3. **Google Cloud Platform (GCP):** IaaS (Compute Engine), PaaS (Cloud SQL, GKE), analytics/ML (BigQuery, Vertex AI).

Service model reminder: IaaS = VMs/networks; PaaS = managed middleware/DB; SaaS = end-user applications.

Quick symbol sheet (Question 1 calculations)

Symbol	Meaning
B	Block size
R	Record length
bf	Records per block (unspanned)
b	Data file blocks
R_i	Index entry length
$bfri$	Index entries per index block
b_1	First-level index blocks

End of document.